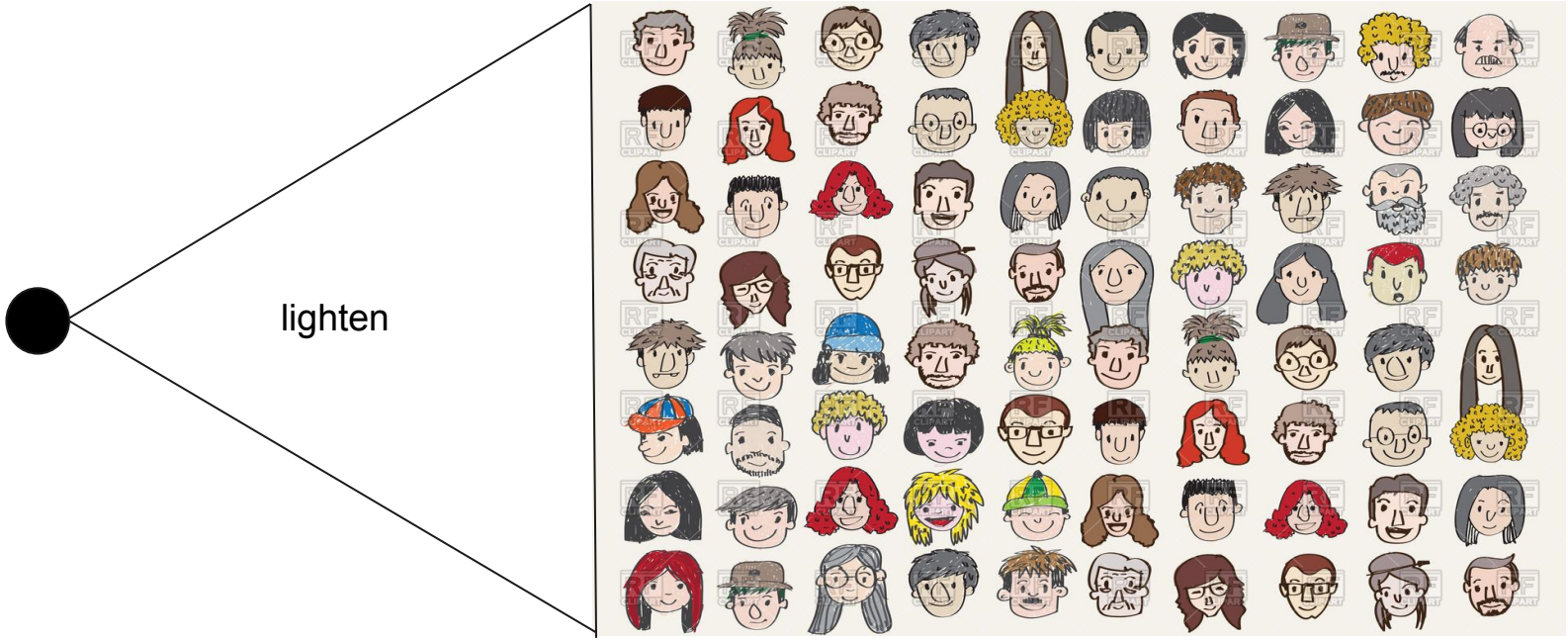
Philosophy behind OdRxObject

By Dheeraj Karki

Motivation



Diversity emerges from a single point called nothing.

What is the difference between ODA and AutoCAD (Development/Design Philosophy)?

OPEN DESIGN ALLIANCE

VS

ODA vs AutoCAD*

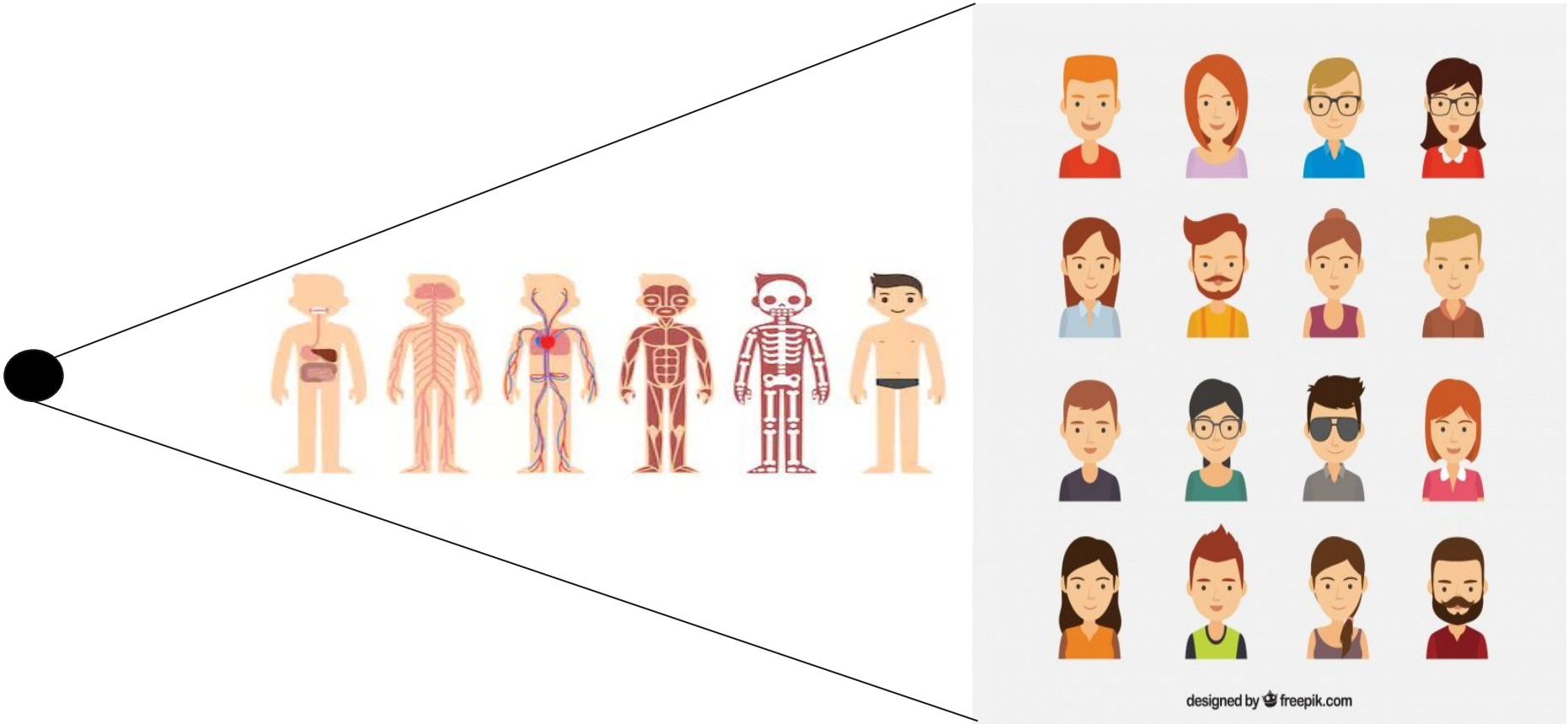
- Focus to support AutoCAD.
- Od used as suffix.
- Used Object smartly(using in-built smart pointer, eg. xyzPtr).

- Focus on new Ideas or Feature
- Ac used as suffix
- Mostly Bare pointer.

* Yes, there might be more difference but We are covering only those difference which is related to ObjectARX.

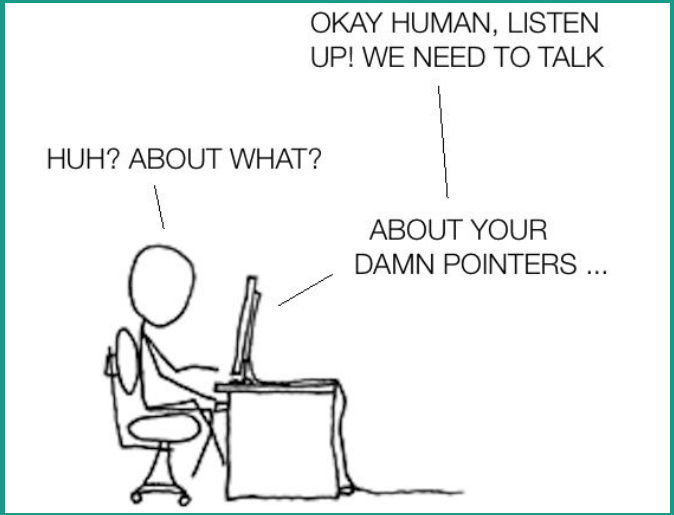
How all are different from each other?

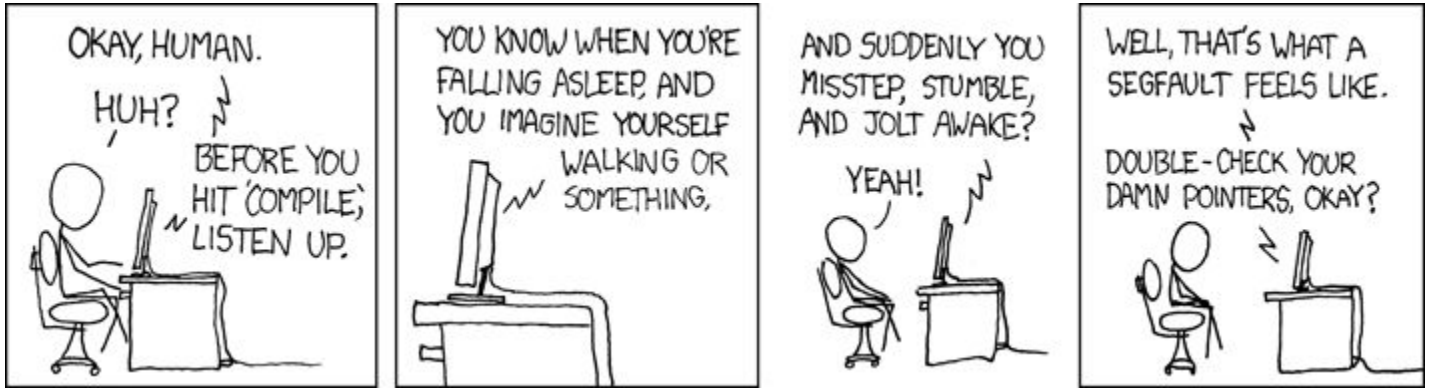
- Open OdaMfcApp with one sample drawing.(Instruction for me)



Similarity in Diversity.

SMART Pointer




OKAY, HUMAN.
HUH?
BEFORE YOU HIT 'COMPILE,' LISTEN UP.
YOU KNOW WHEN YOU'RE FALLING ASLEEP, AND YOU IMAGINE YOURSELF WALKING OR SOMETHING,
AND SUDDENLY YOU MISSTEP, STUMBLE, AND JOLT AWAKE?
YEAH!
WELL, THAT'S WHAT A SEGFAULT FEELS LIKE.
DOUBLE-CHECK YOUR DAMN POINTERS, OKAY?

Problem with Raw Pointer?

- Raw pointers may be fragile and prone to errors, but they're so close to the machine that code using them runs like a bat out of hell. For decades, there was no better way to satisfy the need for speed demanded by a large class of applications. Memory leaks, segfaults, and torturous debugging sessions were simply the price of doing business if you needed the level of performance raw pointers uniquely provided.
- The trouble with raw pointers is that there are too many ways to misuse them, including:
  - forgetting to initialize them,
  - forgetting to release dynamic memory, and
  - releasing dynamic memory too many times.

Things that can go wrong when pointers are misused

Memory leaks - You allocate a pointer in a method and then let it go out of scope without properly deallocating it. The pointer to the memory on the heap is now lost, but the memory remains allocated. Freeing this memory is now extremely difficult.

Access violations - You create a pointer that points at a memory address that you do not have access to, or that does not exist. Pointers are just integers afterall, and can be manipulated like any other number. When you attempt to dereference your invalid pointer, your program will halt.

Things that can go wrong when pointers are misused

Null pointer errors - This is a special case of an access violation. The proper way to "park" a pointer, so that it doesn't point at anything in particular, is to set its value to zero or null. Attempting to dereference a null pointer will halt your program.

Buffer overflows - You allocate a pointer for a character buffer of 30 characters. You then proceed to stream user input (from a socket, file, console, etc.) into this buffer. If you fail to properly implement buffer bounding checks, then your program could potentially put more than 30 characters into the buffer. This will corrupt any data stored adjacent to the buffer in memory and possibly expose you to a malicious code attack.

Things that can go wrong when pointers are misused

Memory corruption - A pointer is just an integer that contains the memory address of something it points to. As an integer, pointer arithmetic can be used to manipulate the pointer's value in all sorts of interesting ways. Subtle bugs can develop if the pointer calculations go wrong. The pointer will now point to some unknown location in memory, and anything could happen when it is dereferenced

Null-terminated string problems - These bugs occur when string library functions that expect null-terminated strings are fed character pointers that are not null terminated. The string library functions will continue to process characters, one at a time, until a null is found -- wherever that may be.

Smart Pointer

Smart pointers are objects that look and feel like pointers, but are smarter. **What does this mean?**

To look and feel like pointers, smart pointers need to have the same interface that pointers do: they need to support pointer operations like dereferencing (operator *) and indirection (operator ->). An object that looks and feels like something else is called a **proxy object**, or **just proxy**. The **proxy pattern** and its many uses are described in the books **Design Patterns** and **Pattern Oriented Software Architecture**.

Why would we use them?

Why: Less Bug

Automatic cleanup. As the code above illustrates, using smart pointers that clean after themselves can save a few lines of code. The importance here is not so much in the keystrokes saved, but in reducing the probability for bugs: you don't need to remember to free the pointer, and so there is no chance you will forget about it.

Automatic initialization. Another nice thing is that you don't need to initialize the auto_ptr to NULL, since the default constructor does that for you. This is one less thing for the programmer to forget.

Why would we use them?

Why: Less Bug

Dangling pointers. A common pitfall of regular pointers is the dangling pointer: a pointer that points to an object that is already deleted. The following code illustrates this situation:

```
MyClass* p(new MyClass);
MyClass* q = p;
delete p;
p->DoSomething();   // Watch out! p is now dangling!
p = NULL;           // p is no longer dangling
q->DoSomething();   // Ouch! q is still dangling!
```

Why would we use them?

Why: Less Bug

Create a new copy of the object pointed by p, and have q point to this copy.

Ownership transfer: Let both p and q point to the same object, but transfer the responsibility for cleaning up ("ownership") from p to q.

The following situations involve transferring ownership from one `unique_ptr` to another: returning from a function, and passing as a parameter to a function like a constructor.

Say you have some polymorphic type `Animal`:

```
struct Animal {
  virtual ~Animal() {}
  virtual void speak() = 0;
};
```

with concrete subclasses `Cat` and `Dog`:

```
struct Cat : Animal {
  void speak() override { std::cout << "Meow!\n"; }
};

struct Dog : Animal {
  void speak() override { std::cout << "Woof!\n"; }
};
```

And you want a simple factory that creates a pet based on a required value of obedience. Then the factory must return a pointer. We want the pet factory to transfer ownership of the created pet to the caller so a reasonable return type is `std::unique_ptr<Animal>`:

```
std::unique_ptr<Animal> createPet(double obedience) {
  if (obedience > 5.0)
    return std::make_unique<Dog>();
  return std::make_unique<Cat>();
}
```

Now, say we want to create a `House` that will own the pet then we might want to pass the pet into the constructor of the `House`. There is some debate (see comments on this blog post) about how best to pass a `unique_ptr` to a constructor but it would look something like this:

```
class House {
 private:
  std::unique_ptr<Animal> pet_;
 public:
  House(std::unique_ptr<Animal> pet) : pet_(std::move(pet)) {}
};
```

We have passed the `unique_ptr` into the constructor and have then "moved" it to the member variable.

The calling code could look something like:

The calling code could look something like:

```
auto pet = createPet(6.0);
House house(std::move(pet));
```

After constructing the `House`, the `pet` variable will be `nullptr` because we have transferred ownership of the pet to the `House`.

Why would we use them?

Why: Less Bug

Reference counting: Maintain a count of the smart pointers that point to the same object, and delete the object when this count becomes zero. So the statement q = p causes the count of the object pointed by p to increase by one.

Why would we use them?

Why: Exception Handling

```
void foo()
{
    MyClass* p(new MyClass);
    p->DoSomething();
    delete p;
}
```

Why would we use them?

Why: Exception Handling

```
void foo()
{
    MyClass* p;
    try {
        p = new MyClass;
        p->DoSomething();
        delete p;
    }
    catch (...) {
        delete p;
        throw;
    }
}
```

Why would we use them?

Why: Garbage Collection

Since C++ does not provide automatic garbage collection like some other languages, smart pointers can be used for that purpose. The simplest garbage collection scheme is reference counting or reference linking, but it is quite possible to implement more sophisticated garbage collection schemes with smart pointers.

Why would we use them?

Why: Efficiency

Smart pointers can be used to make more efficient use of available memory and to shorten allocation and deallocation time.

A common strategy for using memory more efficiently is copy on write (COW). This means that the same object is shared by many COW pointers as long as it is only read and not modified. When some part of the program tries to modify the object ("write"), the COW pointer creates a new copy of the object and modifies this copy instead of the original object. The standard string class is commonly implemented using COW semantics (see the <string> header).

Why would we use them?

Why: Efficiency

```
string s("Hello");

string t = s;          // t and s point to the same buffer of characters

t += " there!";        // a new buffer is allocated for t before
                       // appending " there!", so s is unchanged.
```

Basic Smart Pointer

```
class SP
{
private:
    Person*    pData; // pointer to person class
public:
    SP(Person* pValue) : pData(pValue)
    {
    }
    ~SP()
    {
        // pointer no longer requried
        delete pData;
    }

    Person& operator* ()
    {
        return *pData;
    }

    Person* operator-> ()
    {
        return pData;
    }
};
```

Generic Smart Pointer

```
template < typename T > class SP
{
    private:
    T*    pData; // Generic pointer to be stored
    public:
    SP(T* pValue) : pData(pValue)
    {
    }
    ~SP()
    {
        delete pData;
    }

    T& operator* ()
    {
        return *pData;
    }

    T* operator-> ()
    {
        return pData;
    }
};

void main()
{
    SP<PERSON> p(new Person("Scott", 25));
    p->Display();
    // Dont need to delete Person pointer..
}
```

```
void main()
{
    SP<PERSON> p(new Person("Scott", 25));
    p->Display();
    {
        SP<PERSON> q = p;
        q->Display();
        // Destructor of Q will be called here..
    }
    p->Display();
}
```

Generic Smart Referencing Count Pointer

```cpp
class RC
{
    private:
    int count; // Reference count

    public:
    void AddRef()
    {
        // Increment the reference count
        count++;
    }

    int Release()
    {
        // Decrement the reference count and
        // return the reference count.
        return --count;
    }
};
```

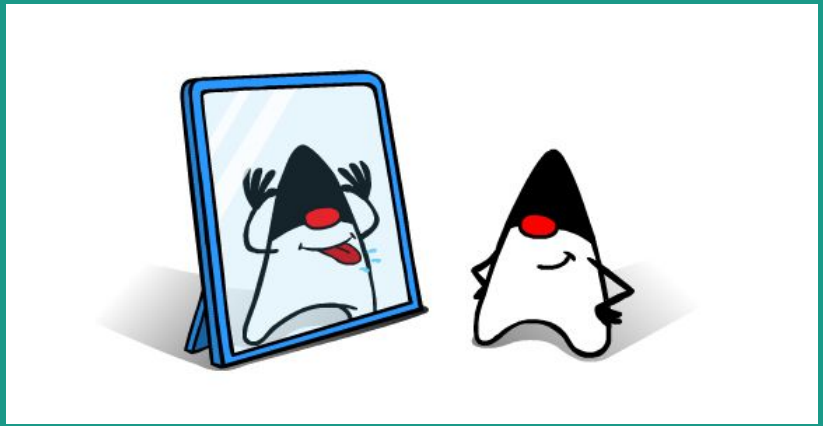
```cpp
template < typename T > class SP
{
private:
    T*    pData;       // pointer
    RC* reference; // Reference count

public:
    SP() : pData(0), reference(0)
    {
        // Create a new reference
        reference = new RC();
        // Increment the reference count
        reference->AddRef();
    }

    SP(T* pValue) : pData(pValue), reference(0)
    {
        // Create a new reference
        reference = new RC();
        // Increment the reference count
        reference->AddRef();
    }

    SP(const SP<T>& sp) : pData(sp.pData), reference(sp.reference
    {
        // Copy constructor
        // Copy the data and reference pointer
        // and increment the reference count
        reference->AddRef();
    }

    ~SP()
    {
        // Destructor
        // Decrement the reference count
        // if reference become zero delete the data
        if(reference->Release() == 0)
        {
            delete pData;
            delete reference;
        }
    }
```

```cpp
    T& operator* ()
    {
        return *pData;
    }

    T* operator-> ()
    {
        return pData;
    }

    SP<T>& operator = (const SP<T>& sp)
    {
        // Assignment operator
        if (this != &sp) // Avoid self assignment
        {
            // Decrement the old reference count
            // if reference become zero delete the old data
            if(reference->Release() == 0)
            {
                delete pData;
                delete reference;
            }

            // Copy the data and reference pointer
            // and increment the reference count
            pData = sp.pData;
            reference = sp.reference;
            reference->AddRef();
        }
        return *this;
    }
};
```

Generic Smart Referencing Count Pointer

```
void main()
{
    SP<PERSON> p(new Person("Scott", 25));
    p->Display();
    {
        SP<PERSON> q = p;
        q->Display();
        // Destructor of q will be called here..

        SP<PERSON> r;
        r = p;
        r->Display();
        // Destructor of r will be called here..
    }
    p->Display();
    // Destructor of p will be called here
    // and person pointer will be deleted
}
```

Reflection



Reflection

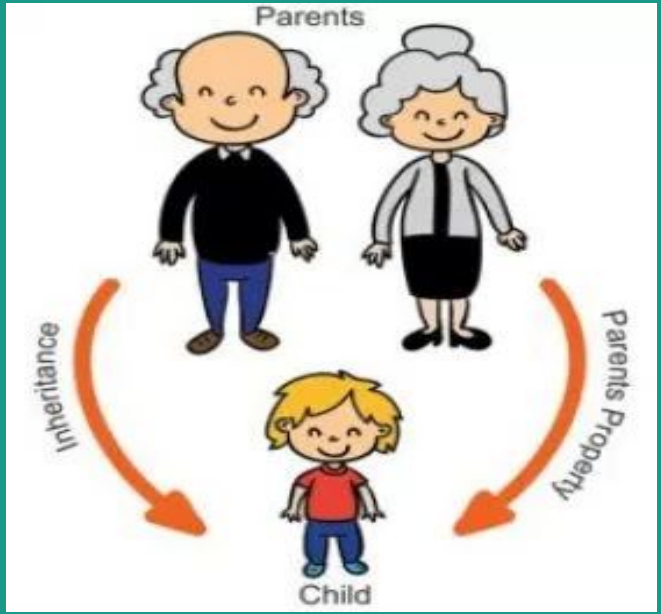
Reflection is a mechanism making it possible to investigate yourself. If we are speaking about programming languages, reflection is used to investigate format of objects at runtime, invoke methods and access fields of these objects. Reflection is required if you want to implement some generic code which can work with objects of unknown (at the time of creation of this code) types. There are several tasks in which reflection can be used: remote method invocation, serialization, object dumps, database interfaces. How it works? Given some object we first should ask for its type. As a result we are given class descriptor, which provides information about class methods and fields. We can use these field descriptors to fetch/store object fields and can use method descriptors to lookup and invoke methods.

Reflection

http://www.garret.ru/cppreflection/docs/reflect.html

http://www.garret.ru/cppreflection/docs/html/hierarchy.html

Inheritance



Inheriting from OdRxObject

- This class is the base class for all OdRxObject objects that require Runtime Type Identification.
- It also provide support for smart pointer (I didn’t say it provide complete feature but I said support).

Let assume you are Software Architect and Designer,
Why will you choose this approach?

Thanks

By Dheeraj Karki